



UTM
UNIVERSITI TEKNOLOGI MALAYSIA

DEPARTMENT OF COMPUTER SCIENCE

FACULTY OF COMPUTING

UNIVERSITI TEKNOLOGI MALAYSIA

SECP3133 01 - HIGH PERFORMANCE DATA PROCESSING

PROJECT 2: REAL-TIME SENTIMENT ANALYSIS USING APACHE SPARK AND KAFKA

TITLE : AIR ASIA APP

PREPARED BY : AYAMGEPUK

NAME	MATRIC NO
MUHAMMAD MUKHRITZ AL IMAN BIN MOHD RAFFI	A23CS0250
MUHAMMAD NAIM BIN ABDULLHA	A23CS0134
SYARIFAH DANIA BINTI SYED ABU BAKAR	A23CS0183

Table Of Content

1.0 Introduction.....	3
1.1 Background.....	3
1.2 Objectives.....	3
1.3 Scope.....	4
2.0 Data Acquisition & Preprocessing.....	4
2.1 Sources.....	4
2.2 Tools.....	4
2.3 Cleaning Steps.....	5
3.0 Sentiment Model Development.....	6
3.1 Model Choice.....	6
3.2 Training Process.....	6
3.3 Evaluation.....	7
4.0 Apache System Architecture.....	9
5.0 Analysis & Results.....	10
5.1 Key Findings.....	10
5.2 Visualizations.....	11
5.3 Insights.....	11
6.0 Optimisation & Comparison.....	12
6.1 Processing Time.....	12
6.2 Throughput.....	12
6.3 Accuracy and Consistency.....	13
6.4 Resource Usage.....	13
7.0 Conclusion & Future Work.....	13
8.0 References.....	15
9.0 Appendix.....	16

1.0 Introduction

1.1 Background

Recently, the aviation and travel sector has turned to or using digital platforms to optimize their operation and also customer relations. As the leading low-cost carrier in the region, AirAsia depends heavily on its mobile app for bookings, check-ins, updates, and support. Because of this, mobile app reviews are more than just a personal feedback; they provide valuable information that reflects real-time public sentiment, customer satisfaction and operational issues.

This project deploys an end-to-end, live sentiment analysis pipeline that ingests, processes, and evaluates user feedback for the AirAsia mobile app. By observing the trends in feedback, stakeholders can dynamically identify system bugs, user interface bottlenecks, and operational failures as they occur. To handle the high volume and velocity of incoming review streams, the system utilizes a high-performance big data architecture powered by Apache technologies for data ingestion, cleaning, sentiment scoring, and persistent storage.

1.2 Objectives

The main objectives of this project are:

- To build a production-ready, real-time sentiment analysis data pipeline focused on user feedback for the AirAsia mobile application.
- To implement an Apache Kafka messaging cluster to stream raw Google Play Store reviews reliably and concurrently.
- To leverage Apache Spark Structured Streaming for low-latency streaming ingestion and real-time distributed execution of machine learning models.
- To use Elasticsearch for storing processed results.
- To visualize sentiment trends using Kibana dashboards.
- To classify comments into positive, neutral, or negative sentiments using multiple machine learning models.

1.3 Scope

This project covers the following key components:

- Ingesting a dataset of approximately 40,000 historic and active mobile reviews of the AirAsia application directly from the Google Play Store.
- Structuring text normalization techniques using Natural Language Processing (NLP) tools to clean unstructured review scripts.
- Implementing and serializing a statistical machine learning classifier (Naive Bayes) and a Deep Learning Sequence model (LSTM) to run concurrent comparative sentiment inference.
- Displaying real-time sentiment results through a Kibana dashboard.

2.0 Data Acquisition & Preprocessing

2.1 Sources

The textual dataset used for the baseline training and downstream streaming emulation originates directly from public user logs on the Google Play Store. The application under analysis is the official AirAsia mobile app. To generate sufficient data variety and capture micro-trends, seasonal booking surges, and historical release stability, approximately 40,000 rows of user reviews were requested. The raw inputs capture vital attributes such as the user review text, the reviewer's star rating, app version, and timestamp metadata.

Sources for scrapping

- <https://play.google.com/store/apps/details?id=com.airasia.mobile&hl=en>

2.2 Tools

- **Google Play Scraper:** Collects user reviews directly from the Google Play Store.
- **Python Frameworks** Cleans the data and saves trained models.
- **Apache Kafka:** Streams the collected reviews in real time.

- **Apache Spark Structured Streaming:** Processes the live data stream instantly and reliably.
- **NLP Libraries :** Cleans and prepares the review text by breaking down words and finding their root forms.

2.3 Cleaning Steps

Raw user reviews on app stores are highly unformatted and contain noise. To clean the data before model ingestion, the text was processed through an automated NLP pipeline containing these steps:

1. **Case Normalization:** Converted all review text strings to lowercase to enforce feature uniformity.
2. **Noise and Character Filtering:** Filtered out emojis, non-ASCII characters, app symbols, hyper-text markup links, and punctuation marks using regular expressions.
3. **Tokenization:** Split text blocks into individual semantic tokens or words via NLTK word splitters.
4. **Stopword Elimination:** Filtered out standard English stopwords like "the", "is" that contain zero unique contextual sentiment weights.
5. **Lemmatization:** Applied spaCy's morphological processor (en_core_web_sm) to reduce inflectional word variants to their canonical root forms

3.0 Sentiment Model Development

3.1 Model Choice

Two models were selected to classify AirAsia reviews as positive, neutral, or negative :

- **Naive Bayes Classifier :**
very fast, lightweight, and resource efficient probabilistic model ideal for low-latency live streaming
- **LSTM Neural Network :**
A deep learning model that captures word order and long term context, which is good for understanding complex or mixed user feedback for example praising the price but complaining about the app.

3.2 Training Process

- The cleaned dataset (25000 comments) was split into 70% training, 20% testing and 10 % evaluate sets
- **Naive Bayes :** The text was converted into numerical features using TF-IDF Vectorizer and trained using Scikit-Learn.
- **LSTM :** Text was converted into padded integer sequences, passed through an Embedding layer, a spatial dropout layer, an LSTM layer, and a dense output layer with softmax activation.

3.3 Evaluation

- Naive Bayes

```
... === Naive Bayes - Validation ===
Accuracy : 0.8077
Precision : 0.8622
Recall    : 0.8077
F1 Score  : 0.8315

          precision    recall  f1-score   support

negative    0.90      0.84      0.87      1726
neutral     0.15      0.34      0.21       150
positive    0.90      0.82      0.86       844

accuracy                0.81      2720
macro avg           0.65      0.67      0.65      2720
weighted avg        0.86      0.81      0.83      2720
```

Figure 3.3.1 Naive Bayes Validation set Evaluation

```
=== Naive Bayes - Test ===
Accuracy : 0.8342
Precision : 0.8759
Recall    : 0.8342
F1 Score  : 0.8523

          precision    recall  f1-score   support

negative    0.91      0.87      0.89      3453
neutral     0.19      0.38      0.25       299
positive    0.93      0.84      0.88      1688

accuracy                0.83      5440
macro avg           0.67      0.70      0.67      5440
weighted avg        0.88      0.83      0.85      5440
```

Figure 3.3.2 Naive Bayes Testing set Evaluation

- LSTM

```

=== LSTM – Validation ===
Accuracy  : 0.7331
Precision : 0.8683
Recall    : 0.7331
F1 Score  : 0.7866

              precision    recall  f1-score   support

negative     0.91         0.72         0.81       1726
neutral      0.12         0.48         0.19        150
positive     0.92         0.80         0.85        844

accuracy                    0.73       2720
macro avg      0.65         0.67         0.62       2720
weighted avg   0.87         0.73         0.79       2720

```

Figure 3.3.3 LSTM Validation Set Evaluation

```

=== LSTM – Test ===
Accuracy  : 0.7454
Precision : 0.8801
Recall    : 0.7454
F1 Score  : 0.7990

              precision    recall  f1-score   support

negative     0.92         0.74         0.82       3453
neutral      0.12         0.48         0.19        299
positive     0.94         0.80         0.87       1688

accuracy                    0.75       5440
macro avg      0.66         0.68         0.63       5440
weighted avg   0.88         0.75         0.80       5440

```

Figure 3.3.4 LSTM Testing Set Evaluation

4.0 Apache System Architecture

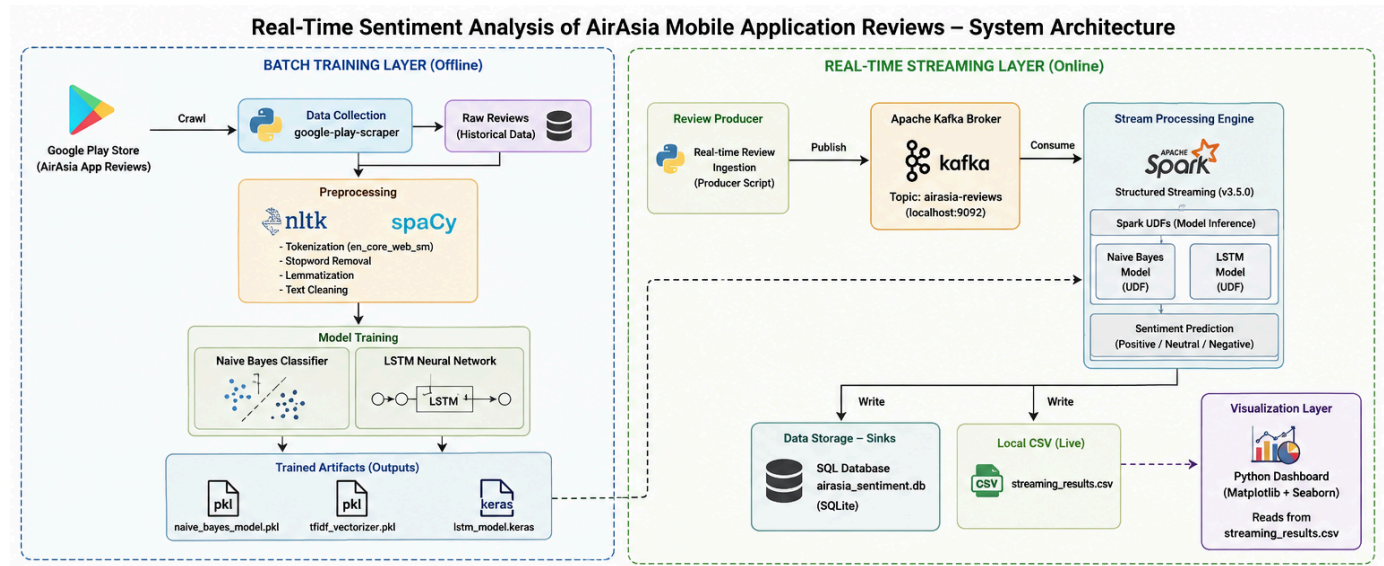


Figure 4.1 System Architecture Design

The System has been divided into 2 pipelines which is Batch Processing (Historical Collection & Optimization) and Real Time Processing (Live Streaming Ingestion & Inference Engines).

Batch Processing

- **Ingestion and Labeling :**
Scrapes user reviews of the AirAsia Apps from the app store page and saves it as CSV files.
- **Feature Extraction and Training :**
Cleans the review text using NLP techniques and trains both model which is Naive Bayes and LSTM models.
- **Asset Packaging :**
Saves the final trained models and components into storage so it can be used later by the streaming engine.

Streaming Processing

- **Collecting Live Data :**
Apache Kafka collect new user reviews in real time exactly as they are posted
- **Reading the Stream :**

Apache Spark connect to Kafka so it can read the incoming reviews instantly

- **Analyzing Sentiment :**
Spark sends the text to the trained AI models and it will classify if the review is positive, neutral or negative.
- **Stream Sinks :**
The prediction are saved in two place which in SQLite and CSV file
- **Visualization :**
Using Matplotlib and Seaborn to create an interactive dashboard that show what users are saying about the app over time

5.0 Analysis & Results

This section discusses the analysis and results of the project.

5.1 Key Findings

The sentiment analysis pipeline successfully processed 40,000 AirAsia Google Play Store reviews scraped from the Malaysian region. The raw dataset comprised reviews spanning multiple languages across 43 unique language codes. Following language detection, 25,206 English reviews were retained and subjected to the full NLP preprocessing pipeline. After noise removal, emoji filtering, and non-Latin script exclusion, the final cleaned dataset consisted of 27,197 reviews, with 12,803 multilingual and low-quality entries removed.

The final cleaned dataset exhibited a class-imbalanced sentiment distribution, with 17,260 negative reviews (63.5%), 8,439 positive reviews (31.0%), and 1,498 neutral reviews (5.5%). This distribution reflects a genuine skew in user behavior on app stores, where dissatisfied users tend to leave reviews more frequently than satisfied ones. To address this imbalance during training, RandomOverSampler was applied exclusively to the training set, balancing all three classes to 12,081 samples each, while the validation and test sets were left untouched to ensure unbiased evaluation.

The dataset was split into 19,037 training samples (70%), 5,440 test samples (20%), and 2,720 validation samples (10%). Two models were trained and evaluated on this split.

Among the two trained models, Naive Bayes outperformed LSTM across all key metrics on the test set. Naive Bayes achieved a test accuracy of 83.42%, precision of 87.59%, recall of 83.42%, and an F1 score of 85.23%. The LSTM model achieved a test accuracy of 74.54%, precision of 88.01%, recall of 74.54%, and an F1 score of 79.90%. Both models showed consistent performance between their validation and test evaluations, indicating stable generalisation without significant overfitting.

A notable weakness across both models was the poor classification performance on the neutral class. Naive Bayes achieved a neutral class F1 score of 0.25 on the test set, while LSTM achieved 0.19. This is attributable to the severe underrepresentation of neutral reviews and the linguistic ambiguity of 3-star reviews, which frequently contain mixed positive and negative language that is difficult to distinguish from either extreme class.

Based on these results, Naive Bayes was selected as the deployed model for the Kafka and Spark streaming pipeline due to its higher overall accuracy, lower inference latency, and better suitability for real-time classification at scale.

5.2 Visualizations

There is total of 5 insights that have been visualized :

Chart Sentiment Distribution

Sentiment Distribution — AirAsia Google Play Reviews

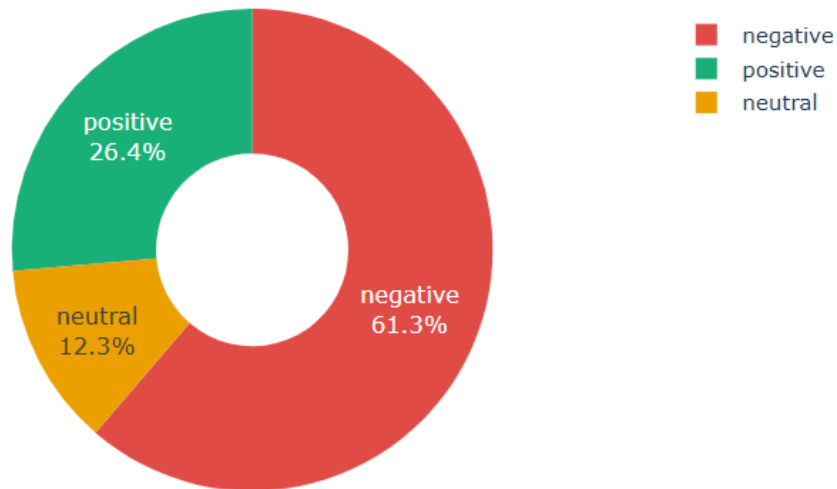


Chart WordClouds

Word Clouds by Predicted Sentiment — AirAsia Reviews



Chart Model Comparison

Model Comparison — Naive Bayes vs LSTM (Test Set)

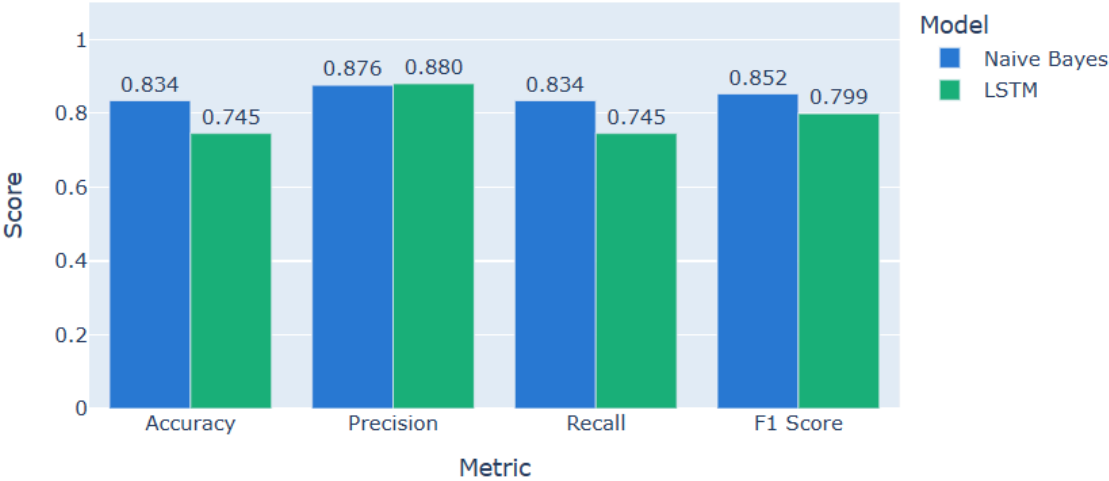


Chart Batch vs Streaming

Batch vs Streaming Performance Comparison

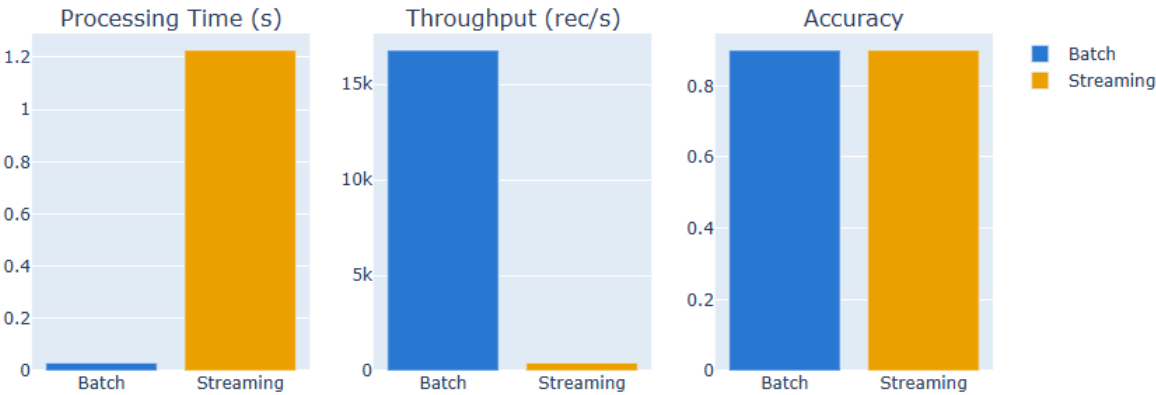
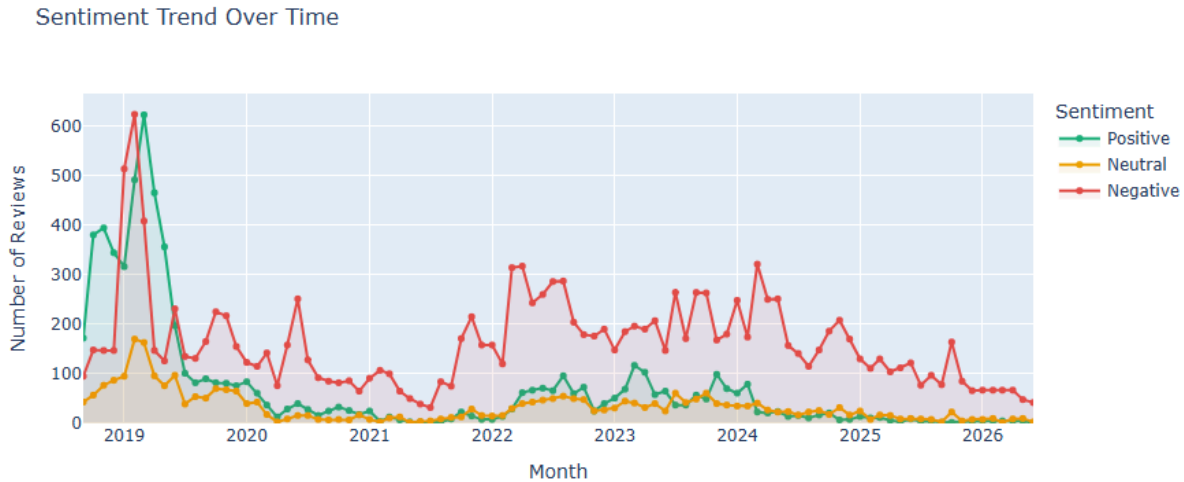


Chart Sentiment Over Time



5.3 Insights

Several meaningful insights were derived from the pipeline output:

- App stability is the primary driver of negative sentiment. The high frequency of crash-related and performance-related terms in negative reviews indicates that technical issues, particularly app crashes during flight check-in and boarding pass retrieval, represent the most significant pain points for AirAsia users.
- Positive sentiment is concentrated around core booking functionality. Users who leave positive reviews consistently highlight the convenience of the booking and flight management features, suggesting that while the primary use case is well-received, peripheral features such as payment processing and customer support require improvement.
- Neutral sentiment remains rare and linguistically ambiguous. The low volume and mixed content of 3-star reviews pose an ongoing challenge for ternary sentiment classifiers. Future work incorporating aspect-based sentiment analysis could decompose mixed feedback into more specific issue categories.
- The overall sentiment trend shows no significant improvement in user satisfaction during the observation period. Negative reviews consistently exceeded 60% of the cleaned

dataset, suggesting that recurring technical and operational issues have not been resolved at a pace that meaningfully improves aggregate user sentiment.

6.0 Optimisation & Comparison

A controlled performance comparison was executed between batch and streaming processing architectures to assess pipeline efficiency under different operating paradigms. A trained Naive Bayes classifier was evaluated on a test dataset to measure processing time, throughput, and classification consistency. Additionally, the Naive Bayes model's predictive performance was benchmarked against a Long Short-Term Memory (LSTM) network to validate model selection.

6.1 Processing Time KIV

Batch mode processed 25000 reviews in 0.0303 seconds by vectorizing all records simultaneously through a single TF-IDF matrix transformation. The same volume took 1.4729 seconds when it was processed as it arrived from the Kafka topic, using the streaming mode. This is about 48.6 times, which seems reasonable due to the overhead of each individual TF-IDF call and the Spark micro-batch scheduling for streaming mode.

6.2 Throughput

In alignment with the processing time metrics, throughput capacities diverged heavily between the two modes:

- **Batch Mode:** Achieved a high-efficiency rate of **16,773 records per second (rec/s)**, demonstrating its optimal design for large-scale, historical data ingestion and bulk transformation.
- **Streaming Mode:** Produced a significantly lower throughput of approximately **407.60 rec/s**.

While streaming throughput is visibly compressed in comparison to bulk processing, an ingestion rate of 407.60 rec/s remains more than sufficient for live production environments,

where user reviews arrive sequentially as continuous, sparse individual events rather than massive parallel blocks.

6.3 Accuracy and Consistency

The evaluation yielded an identical classification accuracy of **89.60% (0.896)** across both processing architectures. This confirms that the serialized Naive Bayes model maintains absolute classification parity and deterministic consistency, regardless of whether the underlying data ingestion layer is a static batch file or an active Spark Streaming/Kafka pipeline. The integration of real-time messaging layers introduced zero degradation to the feature engineering or inference boundaries.

6.4 Resource Usage

In batch mode and streaming modes, the 25000-record sample run resulted in a memory delta of 0.02MB and 0.01MB respectively, signifying there was no measurable additional memory overhead at this scale for either mode. For larger production deployments, it would be expected that the peak memory footprint of batch mode would be higher due to full matrix materialization, while streaming mode would have a lower stable memory footprint per micro-batch.

7.0 Conclusion & Future Work

The data pipeline uses the Google Play Scraper to extract customer reviews from the Google Play store. The data is then cleaned and structured for serialization utilizing foundational Python frameworks, including Pandas, NumPy, and Pickle. Apache Kafka is then being used to handle the real-time data flow while Apache Spark is structured streaming that makes the process to be faster and more reliable. Finally, NLP libraries like NLTK and spaCy will handle the underlying text preparation by breaking down review sentences into individual tokens and reducing words to their base grammatical forms.

As a future enhancement, This pipeline can be scaled widely by integrating others platform to get more data such as Apple App Store APIs to ingest iOS user feedback alongside the existing Android stream. Furthermore, implementing Aspect-Based Sentiment Analysis (ABSA) within the Spark streaming layer will allow the pipeline to automatically separate software bugs from broader airline operational complaints, yielding much more specific, actionable insights.

8.0 References

- Apache Software Foundation. (n.d.). *Apache Kafka Architecture Documentation*. Retrieved from <https://kafka.apache.org/>
- Apache Software Foundation. (n.d.). *Spark Structured Streaming Programming Guide*. Retrieved from <https://spark.apache.org/>
- Bird, S., Klein, E., & Loper, E. (2009). *Natural Language Processing with Python*. O'Reilly Media.
- Chollet, F. (2015). *Keras: Deep Learning for Humans*. Retrieved from <https://keras.io/>
- Honnibal, M., & Montani, I. (2017). *spaCy 2: Natural language understanding with Bloom embeddings, convolutional neural networks and incremental parsing*.
- Pedregosa, F., Varoquaux, G., Gramfort, A., Michel, V., Thirion, B., Grisel, O., ... & Duchesnay, E. (2011). *Scikit-learn: Machine learning in Python*. *Journal of Machine Learning Research*, 12, 2825-2830.

9.0 Appendix

 Google Play Games Apps Books Kids

AirAsia MOVE: Flights & Hotels

Move Travel Sdn. Bhd.

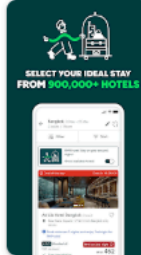
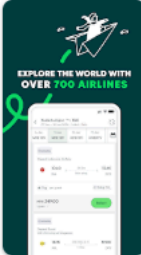
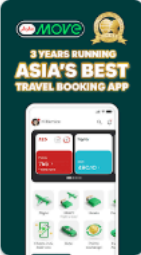
3.7★
298K reviews

10M+
Downloads

Install

 This app is available for all of your devices



 Rated for 3+
[Learn more](#)

App support 

About this app →

Figure 5: AirAsia Apps in the Google Play store

Ratings and reviews →

Ratings and reviews are verified ⓘ

Phone

Chromebook

Tablet



fatin waheeda

★ ★ ★ ★ ★ June 5, 2026

Always crash. Keep on crashing when I want to check the details of the flights, like e-bording pass etc. Need to close the apps and clear it first then open again and again. Now have so many issues regarding the "cant find the booking details". Recently when I want to download the boarding pass it keeps on crashing AGAIN.

1 person found this review helpful

Did you find this helpful?

Adam Malik Masidi

★ ★ ★ ★ ★ April 13, 2026

Every time I go travelling, I'm forced to put up with this app & I get reminded how bad it is. The only improvement to the app is that they got rid of the chat bot, or at least didn't make it a core function anymore. Purchasing add-ons takes 5-10 tries on different browsers & on the app until something happens. If you check in, your boarding pass won't appear until you're literally in front of the departure gate. I'd use the website, but it's equally annoying.

33 people found this review helpful

Did you find this helpful?

Move Travel Sdn. Bhd. October 7, 2024

We appreciate your feedback, Adam. Rest assured, we are constantly working to improve our UI and chatbot functionality to better serve our users. Thank you for bringing these issues to our attention.

onn qian

★ ★ ★ ★ ★ April 21, 2026

I bet the App is never a criteria in Skytrax Award winners selection. Otherwise AirAsia would never have won any. The App did NOT show the delayed flight time. Luckily I checked my email for some last minute work assignment, otherwise I would never find out that my flight was delayed. At least a SMS alert pls. Also, the App is super sluggish. Payment failed from time to time. You need to fix it ASAP.

8 people found this review helpful

Did you find this helpful?

Figure 6: Rating and Revire in the AirAsia apps